

Progreso

9 de abril de 2026

Índice

1. Módulos del sistema	2
2. Desarrollo módulo por módulo	2
2.1. Módulo de orquestación: <code>src/main.py</code>	2
2.2. Módulo 1: <code>src/1_equation_definition</code>	3
2.3. Módulo 2: <code>src/2_parameter_grid</code>	3
2.4. Módulo 3: <code>src/3_zero_finding</code>	3
2.5. Módulo 4: <code>src/4_data_preparation</code>	4
2.6. Módulo 5: <code>src/5_symbolic_regression</code>	5
2.7. Módulo 6: <code>src/6_expression_builder</code>	10
3. Benchmark del pipeline	10
3.1. Catálogo de casos y respuesta esperada	10
3.2. Propiedad de no cruce de ramas	17
3.3. Resultado observado con parámetros actuales	17
3.4. Pendiente experimental	17

1. Módulos del sistema

El pipeline queda organizado en siete bloques de trabajo:

1. **Módulo de orquestación:** `src/main.py`.
2. **Módulo 1 (definición simbólica):** `src/1_equation_definition`.
3. **Módulo 2 (muestreo de parámetros):** `src/2_parameter_grid`.
4. **Módulo 3 (resolución de ceros):** `src/3_zero_finding`.
5. **Módulo 4 (preparación de datos):** `src/4_data_preparation`.
6. **Módulo 5 (regresión simbólica):** `src/5_symbolic_regression`.
7. **Módulo 6 (reporte de funciones encontradas):** `src/6_expression_builder`.

2. Desarrollo módulo por módulo

2.1. Módulo de orquestación: `src/main.py`

Perspectiva teórica

Desde el punto de vista teórico, este módulo define la composición del pipeline como una transformación secuencial:

$$\mathcal{E} \xrightarrow{\text{parseo}} \mathcal{S} \xrightarrow{\text{muestreo}} \Theta \xrightarrow{\text{solver}} \mathcal{D} \xrightarrow{\text{preparacion}} \mathcal{D}_{SR} \xrightarrow{SR} \mathcal{G} \xrightarrow{\text{postproceso}} \mathcal{G}^*$$

donde:

- $\mathcal{E}$ es la ecuación de entrada,
- Θ es el conjunto de tuplas de parámetros,
- $\mathcal{D}$ son los pares observacionales $(params, roots)$,
- $\mathcal{G}$ son expresiones candidatas descubiertas,
- $\mathcal{G}^*$ es la salida final reportada.

Además, este módulo fija reproducibilidad experimental porque concentra la lectura de configuración, el orden de etapas y la persistencia de artefactos.

Archivos

- `src/main.py`:
 - valida modo `SR_INPUT_MODE`,
 - ejecuta pasos 1 a 6 en orden,
 - soporta flujo `combined` y `branches`,
 - guarda `config.json`, `results_summary.json` y `final_expressions.txt`.

2.2. Módulo 1: src/1_equation_definition

Perspectiva teórica

Este módulo formaliza la ecuación paramétrica de entrada

$$F(x; \alpha_1, \dots, \alpha_n) = 0$$

en una representación simbólica manipulable. La idea teórica clave es desacoplar la sintaxis textual del problema matemático: se transforma un string en un objeto algebraico para derivar, sustituir y resolver de manera sistemática.

Archivos

- `src/1_equation_definition/equation_parser.py`:
 - `parse_equation(...)` crea símbolos SymPy para variables y parámetros, y parsea la ecuación con `sympify`.
 - `get_equation_info(...)` genera metadatos (formato latex, complejidad aproximada, símbolos).

2.3. Módulo 2: src/2_parameter_grid

Perspectiva teórica

Este módulo construye el dominio discreto de evaluación de parámetros. Sea cada parámetro $\alpha_i \in [m_i, M_i]$ con k_i puntos, entonces el conjunto total es el producto cartesiano:

$$\Theta = \prod_{i=1}^n \{\alpha_i^{(1)}, \dots, \alpha_i^{(k_i)}\}$$

y su cardinalidad es $|\Theta| = \prod_i k_i$. Este paso controla la cobertura del espacio paramétrico y el costo computacional total del pipeline.

Archivos

- `src/2_parameter_grid/grid_generator.py`:
 - `generate_grid(...)` discretiza rangos con `numpy.linspace`.
 - calcula el producto cartesiano con `itertools.product`.
 - retorna `grid` en forma matricial y el orden de parámetros.

2.4. Módulo 3: src/3_zero_finding

Perspectiva teórica

Este módulo resuelve, para cada $\theta \in \Theta$, la ecuación escalar:

$$F(x; \theta) = 0$$

y obtiene un conjunto de raíces reales. Teóricamente, transforma el problema simbólico en un dataset supervisado multivaluado:

$$\mathcal{D} = \{(\theta_k, r_{k,1}), (\theta_k, r_{k,2}), \dots\}$$

con filtrado de soluciones no reales y normalización de orden.

Como línea de trabajo futura, esta etapa se evaluará también con herramientas de `SciPy` para comparar robustez numérica y costo computacional frente al enfoque actual basado en `SymPy`.

Archivos

- `src/3_zero_finding/solver.py`:
 - `solve_equation_for_parameters(...)` resuelve una tupla vía `solve` o `nsolve`.
 - `_nsolve_multiple(...)` prueba varios guesses para aproximar múltiples raíces con `nsolve`.
 - `solve_for_all_parameter_tuples(...)` recorre todo el grid y acumula resultados válidos.
 - filtra complejos con tolerancia y opcionalmente ordena las raíces.

2.5. Módulo 4: `src/4_data_preparation`

Perspectiva teórica

Este módulo decide cómo representar la relación multivaluada parámetro-raíz antes de la regresión simbólica:

- **en ramas**: descompone en subproblemas univaluados por índice ordinal de raíz,
- **combinado**: conserva todas las raíces en un único conjunto y delega la separación al algoritmo iterativo de SR.

La elección afecta el tipo de hipótesis que aprende el modelo: funciones por rama explícitas o cobertura secuencial de un conjunto mixto.

Archivos

- `src/4_data_preparation/root_grouping.py`:
 - `expand_tuples(...)` expande cada tupla en filas (*params*, *root*).
 - `group_by_root_branch(...)` agrupa por rama ordinal.
 - `combine_all_roots(...)` crea dataset unico para modo combinado.

2.6. Módulo 5: `src/5_symbolic_regression`

Perspectiva teórica

Este es el núcleo del sistema. El objetivo es encontrar expresiones simbólicas $g(\theta)$ que minimicen la discrepancia con los datos observados. El enfoque implementado es de cobertura iterativa:

1. entrenar una corrida de SR sobre puntos restantes,
2. evaluar candidatas del Hall of Fame,
3. seleccionar la expresión con mayor cobertura,
4. remover puntos cubiertos,
5. repetir hasta condicion de parada.

Conceptualmente, es un esquema voraz sobre conjuntos de puntos cubiertos por expresiones candidatas. La calidad depende de la función de pérdida, los operadores permitidos, las restricciones estructurales y el criterio de match.

Parámetros activos actuales del modelo (PySR)

En esta subsección se explicita, para cada parámetro de entrenamiento, el valor por defecto de PySR y el valor adoptado en el proyecto.

Los valores por defecto se verificaron desde la instancia base de `PySRRegressor()` en el entorno actual.

Parametro	Default PySR	Valor proyecto	Motivo de uso
<code>niterations</code>	100	500	Aumentar la profundidad de búsqueda para mejorar la recuperación de formas analíticas complejas.
<code>populations</code>	31	30	Mantener estrategia multiisla con alta diversidad estructural y costo controlado.
<code>population_size</code>	27	50	Incrementar cobertura de candidatos por isla para reducir convergencia prematura.
<code>ncycles_per_iteration</code>	380	550	Favorecer un refinamiento local más largo antes de la siguiente migración.
<code>maxsize</code>	30	25	Acotar complejidad para evitar expresiones excesivas sin perder capacidad expresiva necesaria.

<code>parsimony</code>	<code>0.0</code>	<code>0.0</code>	Se mantiene default; el control de complejidad se hace con costos por operador y restricciones.
<code>model_selection</code>	<code>best</code>	<code>best</code>	Se mantiene default por equilibrio entre error y complejidad en Hall of Fame.
<code>turbo</code>	<code>False</code>	<code>True</code>	Activar la ruta acelerada de evaluación para reducir el tiempo de corrida.
<code>procs</code>	<code>None</code>	<code>0</code>	Forzar una configuración conservadora de paralelismo para la estabilidad de RAM en el entorno local.
<code>progress</code>	<code>True</code>	<code>bool(VERBOSE)</code>	Acoplar salida de progreso al nivel de verbosidad global del proyecto.
<code>verbosity</code>	<code>1</code>	<code>1 u 0</code>	Ajustar detalle de logs segun ejecucion detallada o silenciosa.
<code>temp_equation_file</code>	<code>False</code>	<code>True</code>	Mantener trazabilidad de ecuaciones temporales durante la corrida.
<code>delete_tempfiles</code>	<code>True</code>	<code>True</code>	Se mantiene default para limpiar temporales al finalizar.

Qué significa cada parámetro de la tabla

- **niterations:** número de iteraciones internas de evolución que ejecuta PySR en una corrida.
- **populations:** cantidad de poblaciones independientes (islas evolutivas) que se mantienen en paralelo.
- **population_size:** cantidad de individuos (ecuaciones candidatas) dentro de cada población.
- **ncycles_per_iteration:** cantidad de ciclos evolutivos locales que se ejecutan antes de sincronización/migración.
- **maxsize:** tamaño máximo permitido del árbol simbólico (complejidad estructural de una ecuación).
- **parsimony:** peso de penalización por complejidad en la función objetivo de búsqueda.
- **model_selection:** regla para elegir la ecuación final desde el Hall of Fame.
- **turbo:** bandera de optimización de evaluación numérica acelerada en backend Julia.

- `procs`: número de procesos de Julia usados para paralelismo de evaluación.
- `progress`: controla si se muestra barra de progreso durante entrenamiento.
- `verbosity`: nivel de detalle de logs en consola.
- `temp_equation_file`: indica si se escribe archivo temporal con ecuaciones intermedias.
- `delete_tempfiles`: indica si esos archivos temporales se eliminan automáticamente al terminar.

Operadores y restricciones activas

La gramática simbólica actual incorpora operadores base y operadores custom con sesgo de dominio:

- Operadores unarios: base `UNARY_OPERATORS` con `neg` + `safe_sqrt`, `safe_cbrt`, `safe_root4`, ..., `safe_root10`.
- Operadores binarios: base `BINARY_OPERATORS` con `+`, `-`, `*`, `/` + `safe_pow`.

La motivación técnica es estabilidad numérica e interpretabilidad:

- `safe_pow` preserva el signo para bases negativas y evita comportamientos no reales en potencias fraccionarias.
- Se penaliza `safe_pow` con complejidad 4 para favorecer formas más explicables cuando existe una alternativa en términos de raíces explícitas.
- En la práctica, con la configuración de búsqueda actual (iteraciones/poblaciones/ciclos), cuando se intentaba expresar raíces solo a través de `safe_pow` la convergencia no era estable.
- Por esa razón se decidió explicitar operadores de raíz `safe_sqrt`, `safe_cbrt`, `safe_root4`, ..., `safe_root10`; con esta gramática la convergencia observada fue consistente en los casos de trabajo.
- Se aplican restricciones de anidamiento para evitar explosiones combinatorias (por ejemplo, no permitir anidamiento arbitrario entre operadores de raíz).
- Se agrega `constraints[safe_pow] = (9, 1)` para controlar complejidad de argumentos en potencia segura.

Funciones de pérdida y criterio de match

Las funciones de pérdida definen **qué significa "mejor"** durante el entrenamiento evolutivo. En este proyecto hay tres modos, implementados en `src/5_symbolic_regression/loss_functions.py` e inyectados al modelo desde `symbolic_regression.py`.

1) MSE (error cuadrático medio, modo base) Es la pérdida por defecto del backend de SymbolicRegression.jl cuando no se activa ninguna pérdida custom:

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Interpretación:

- penaliza fuertemente errores grandes (cuadrático),
- favorece un buen ajuste promedio global,
- sirve como baseline estándar para comparar contra las otras pérdidas.

2) Pérdida sigmoideal (umbral suave) Se implementa por punto como:

$$L_{sig}(y, \hat{y}) = \frac{1}{1 + \exp(-k(|y - \hat{y}| - \varepsilon))}$$

y la pérdida de la expresión es el promedio de esos valores punto a punto.

Interpretación:

- si $|y - \hat{y}| < \varepsilon$, la pérdida tiende a 0,
- si $|y - \hat{y}| > \varepsilon$, la pérdida tiende a 1,
- ε define la banda de tolerancia y k define qué tan abrupta es la transición.

3) Pérdida de conteo duro de matches Se implementa por punto como una función indicadora:

$$L_{hard}(y, \hat{y}) = \mathbf{1}(|y - \hat{y}| \geq \varepsilon_{mc})$$

y al promediar en el dataset se obtiene la fracción de no-matches.

Interpretación:

- optimiza directamente la cobertura (maximizar matches),
- no diferencia cuánto se falla, solo si matchea o no,
- alinea el entrenamiento con el objetivo final de cubrir la mayor cantidad de puntos.

Activación práctica de cada modo

- **MSE (actual por defecto):** `USE_SIGMOID_LOSS = False` y `USE_MATCH_COUNT_LOSS = False`.
- **Sigmoideal:** `USE_SIGMOID_LOSS = True`.
- **Conteo duro:** `USE_MATCH_COUNT_LOSS = True` y `MATCH_COUNT_EPSILON = 1e-4`.

Regla de consistencia: no se permite activar simultáneamente `USE_SIGMOID_LOSS` y `USE_MATCH_COUNT_LOSS`.

Relación entre pérdida de entrenamiento y criterio de match Además de la pérdida de entrenamiento, el proyecto aplica un criterio de match para medir cobertura al evaluar ecuaciones candidatas:

- para MSE y sigmoidal se usa match **relativo**: $|y - \hat{y}| < \varepsilon(1 + |y|)$ con `EPSILON` = 0.005,
- para conteo duro se usa match **absoluto**: $|y - \hat{y}| < \varepsilon_{mc}$.

Esta separación entre "pérdida para entrenar" y regla de match para cobertura.^{es} clave para interpretar los resultados del algoritmo iterativo.

Características del algoritmo iterativo

El algoritmo del wrapper externo (función `iterative_symbolic_regression`) trabaja sobre puntos remanentes y produce una secuencia de ecuaciones.

Qué significa una iteración en este proyecto Una iteración externa corresponde a:

1. correr **una** vez PySR sobre los puntos restantes,
2. evaluar todo el Hall of Fame,
3. elegir la ecuación con mayor cantidad de matches,
4. si pasa el umbral mínimo, remover sus puntos y continuar.

Por eso hay dos escalas de iteración:

- **iteración interna de PySR**: `niterations` = 500,
- **iteración externa del wrapper**: contador `iteration` del algoritmo iterativo.

Costo computacional En primera aproximación,

costo total $\propto I_{ext} \times C_{PySR}(niterations, populations, population_size, ncycles_per_iteration)$

donde I_{ext} es el número de iteraciones externas aceptadas/reintentadas.

Condiciones de parada actualmente vigentes

El loop externo se detiene por cualquiera de estas condiciones:

1. **Pocos puntos restantes**: si `len(X_remaining)` < `MIN_POINTS`. Hoy: `MIN_POINTS` = 5.
2. **Máximo de iteraciones externas**: si `MAX_ITERATIONS` no es `None` y se supera. Hoy: `MAX_ITERATIONS` = `None` (sin tope fijo).
3. **Estancamiento por cero match**: si hay `MAX_CONSECUTIVE_NO_MATCH` iteraciones seguidas sin matches. Hoy: `MAX_CONSECUTIVE_NO_MATCH` = 3.
4. **Estancamiento por baja cobertura**: si la mejor ecuación de la iteración queda por debajo de `MIN_MATCH_FRACTION` y eso se repite consecutivamente hasta el mismo límite. Hoy: `MIN_MATCH_FRACTION` = 0.05.

Archivos

- `src/5_symbolic_regression/symbolic_regression.py`:
 - construye y entrena `PySRRegressor` con operadores base y custom seguros,
 - define mappings a SymPy y restricciones de anidamiento,
 - evalúa Hall of Fame para maximizar puntos matcheados,
 - implementa el algoritmo iterativo y sus condiciones de parada.
- `src/5_symbolic_regression/loss_functions.py`:
 - define `sigmoid_loss` y `match_count_loss` en Python,
 - expone versiones equivalentes en sintaxis Julia para PySR.
- `src/5_symbolic_regression/utils.py`:
 - define criterio de match relativo/absoluto,
 - provee funciones de trazabilidad por iteración.
- `src/5_symbolic_regression/regression_adapter.py`:
 - interfaz de ejecución para modo por ramas y modo combinado,
 - desacopla el consumo de datos del motor iterativo.

2.7. Módulo 6: `src/6_expression_builder`

Perspectiva teórica

Este módulo estandariza la salida de la etapa de regresión simbólica para reportar, de forma explícita, las funciones descubiertas por rama.

También aporta utilidades de resumen orientadas a reporte, preservando la trazabilidad entre ecuación encontrada y cobertura de puntos.

Archivos

- `src/6_expression_builder/functions_report.py`:
 - `return_discovered_functions(...)` devuelve sin postproceso las funciones encontradas.
 - `summarize_discovered_functions(...)` genera un resumen liviano con índice, ecuación y puntos matcheados.

3. Benchmark del pipeline

3.1. Catálogo de casos y respuesta esperada

El benchmark actual del proyecto usa 43 casos. Cada uno fija una ecuación, rangos de parámetros y una lista ordenada de raíces esperadas (una por rama real).

Listado completo por categoría

Lineales (casos 1–10)

- linear_01_basic

Ecuación:

$$x + a - 2 = 0$$

Ramas esperadas:

$$[2 - a]$$

- linear_02_scaled

Ecuación:

$$2x - a = 0$$

Ramas esperadas:

$$\left[\frac{a}{2}\right]$$

- linear_03_offset

Ecuación:

$$x - 3a + 5 = 0$$

Ramas esperadas:

$$[3a - 5]$$

- linear_04_fraction

Ecuación:

$$3x + a - 6 = 0$$

Ramas esperadas:

$$\left[\frac{6-a}{3}\right]$$

- linear_05_negative

Ecuación:

$$-x + a + 1 = 0$$

Ramas esperadas:

$$[a + 1]$$

- linear_06_two_params

Ecuación:

$$x - a - b = 0$$

Ramas esperadas:

$$[a + b]$$

- linear_07_scaled_params

Ecuación:

$$x - 2a + 3b = 0$$

Ramas esperadas:

$$[2a - 3b]$$

- linear_08_product_param

Ecuación:

$$x - ab = 0$$

Ramas esperadas:

$$[ab]$$

- linear_09_division

Ecuación:

$$ax - b = 0$$

Ramas esperadas:

$$\left[\frac{b}{a}\right]$$

- linear_10_three_params

Ecuación:

$$x - a - b - c = 0$$

Ramas esperadas:

$$[a + b + c]$$

Cuadráticas (casos 11–24)

- quadratic_11_simple

Ecuación:

$$x^2 - a = 0$$

Ramas esperadas:

$$[-\sqrt{a}, \sqrt{a}]$$

- quadratic_12_shifted

Ecuación:

$$x^2 - 2ax + a^2 = 0$$

Ramas esperadas:

$$[a]$$

- quadratic_13_factored

Ecuación:

$$(x - a)(x + a) = 0$$

Ramas esperadas:

$$[-a, a]$$

- quadratic_14_linear_coeff

Ecuación:

$$x^2 + ax = 0$$

Ramas esperadas:

$$[-a, 0]$$

- quadratic_15_full
Ecuación:
 $x^2 - 3ax + 2a^2 = 0$
Ramas esperadas:
 $[a, 2a]$
- quadratic_16_irrational
Ecuación:
 $x^2 - 2x - a = 0$
Ramas esperadas:
 $[1 - \sqrt{1 + a}, 1 + \sqrt{1 + a}]$
- quadratic_17_param_coeff
Ecuación:
 $ax^2 - 1 = 0$
Ramas esperadas:
 $[-\frac{1}{\sqrt{a}}, \frac{1}{\sqrt{a}}]$
- quadratic_18_two_params
Ecuación:
 $x^2 - ax - b = 0$
Ramas esperadas:
 $[\frac{a - \sqrt{a^2 + 4b}}{2}, \frac{a + \sqrt{a^2 + 4b}}{2}]$
- quadratic_19_full_abc
Ecuación:
 $ax^2 + bx + c = 0$
Ramas esperadas:
 $[\frac{-b - \sqrt{b^2 - 4ac}}{2a}, \frac{-b + \sqrt{b^2 - 4ac}}{2a}]$
- quadratic_20_factored_two
Ecuación:
 $(x - a)(x - b) = 0$
Ramas esperadas:
 $[b, a]$
- quadratic_21_sum_product
Ecuación:
 $x^2 - (a + b)x + ab = 0$
Ramas esperadas:
 $[b, a]$
- quadratic_22_symmetric
Ecuación:
 $x^2 - a^2 - b^2 = 0$

Ramas esperadas:

$$[-\sqrt{a^2 + b^2}, \sqrt{a^2 + b^2}]$$

- quadratic_23_nested

Ecuación:

$$x^2 - 4ab = 0$$

Ramas esperadas:

$$[-2\sqrt{ab}, 2\sqrt{ab}]$$

- quadratic_24_depressed

Ecuación:

$$x^2 + ax + \frac{a^2}{4} = 0$$

Ramas esperadas:

$$[-\frac{a}{2}]$$

Cúbicas (casos 25–30)

- cubic_25_one_root

Ecuación:

$$x^3 - a = 0$$

Ramas esperadas:

$$[a^{1/3}]$$

- cubic_26_factored

Ecuación:

$$x(x - a)(x + a) = 0$$

Ramas esperadas:

$$[-a, 0, a]$$

- cubic_27_quadratic_factor

Ecuación:

$$(x - a)(x^2 + 1) = 0$$

Ramas esperadas:

$$[a]$$

- cubic_28_depressed

Ecuación:

$$x^3 - 3ax + 2a^{3/2} = 0$$

Ramas esperadas:

$$[-2\sqrt{a}, \sqrt{a}]$$

- cubic_29_two_params

Ecuación:

$$x^3 - abx = 0$$

Ramas esperadas:

$$[-\sqrt{ab}, 0, \sqrt{ab}]$$

- cubic_30_triple

Ecuación:

$$(x - a)^3 = 0$$

Ramas esperadas:

$$[a]$$

Cuárticas y quíntica (casos 31–35)

- quartic_31_biquadratic

Ecuación:

$$x^4 - a^2 = 0$$

Ramas esperadas:

$$[-\sqrt{a}, \sqrt{a}]$$

- quartic_32_factored

Ecuación:

$$(x - a)(x + a)(x - 2a)(x + 2a) = 0$$

Ramas esperadas:

$$[-2a, -a, a, 2a]$$

- quartic_33_double_quadratic

Ecuación:

$$(x^2 - a)(x^2 - b) = 0$$

Ramas esperadas:

$$[-\sqrt{b}, -\sqrt{a}, \sqrt{a}, \sqrt{b}]$$

- quintic_34_factored

Ecuación:

$$x(x - a)(x + a)(x - 2a)(x + 2a) = 0$$

Ramas esperadas:

$$[-2a, -a, 0, a, 2a]$$

- quartic_35_repeated

Ecuación:

$$(x - a)^2(x + a)^2 = 0$$

Ramas esperadas:

$$[-a, a]$$

Especiales (casos 36–43)

- special_36_reciprocal

Ecuación:

$$ax - 1 = 0$$

Ramas esperadas:

$$\left[\frac{1}{a}\right]$$

- special_37_ratio

Ecuación:

$$ax - b = 0$$

Ramas esperadas:

$$\left[\frac{b}{a}\right]$$

- special_38_quadratic_ratio

Ecuación:

$$x^2 - \frac{a}{b} = 0$$

Ramas esperadas:

$$\left[-\sqrt{\frac{a}{b}}, \sqrt{\frac{a}{b}}\right]$$

- special_39_difference_of_squares

Ecuación:

$$x^2 - (a - b)^2 = 0$$

Ramas esperadas:

$$\left[-(a - b), a - b\right]$$

- special_40_linear_three_params

Ecuación:

$$ax + bx - c = 0$$

Ramas esperadas:

$$\left[\frac{c}{a+b}\right]$$

- special_41_square_root_form

Ecuación:

$$x^2 - (a + b)^2 = 0$$

Ramas esperadas:

$$\left[-(a + b), a + b\right]$$

- special_42_cubic_simple

Ecuación:

$$x^3 - a^3 = 0$$

Ramas esperadas:

$$[a]$$

- special_43_mixed

Ecuación:

$$x^2 - ax - bx + ab = 0$$

Ramas esperadas:

$$[b, a]$$

3.2. Propiedad de no cruce de ramas

En estos casos de prueba, las ramas no se cruzan. El catálogo se construyó con rangos de parámetros que conservan el orden de las raíces esperadas en cada caso multirrama (de menor a mayor), evitando intercambios de etiqueta entre rama 1, rama 2, etc.

3.3. Resultado observado con parámetros actuales

Con los parámetros actuales de SR del proyecto, usando pérdida MSE y modo `branches`, la batería completa de 43 casos registró éxito de pipeline en todos los casos.

3.4. Pendiente experimental

Queda pendiente ejecutar estos mismos 43 casos en modo `combined` para cada una de las tres expresiones de pérdida:

1. MSE,
2. sigmoidal,
3. match count (conteo duro).